Deploying a Dockerized Flask API + Streamlit Frontend in GitHub Codespaces

This guide walks you through deploying a two-container application in GitHub Codespaces using Docker. The project has a Flask-based REST API as the backend and a Streamlit web app as the frontend. Both services are containerized with their own Dockerfiles, connected through a Docker bridge network, and exposed through Codespaces port forwarding.

The example used here is a Telecom Customer Churn Prediction App. The backend serves predictions through a REST API (powered by a trained ML model), and the frontend provides a user-friendly interface for both single and batch predictions.

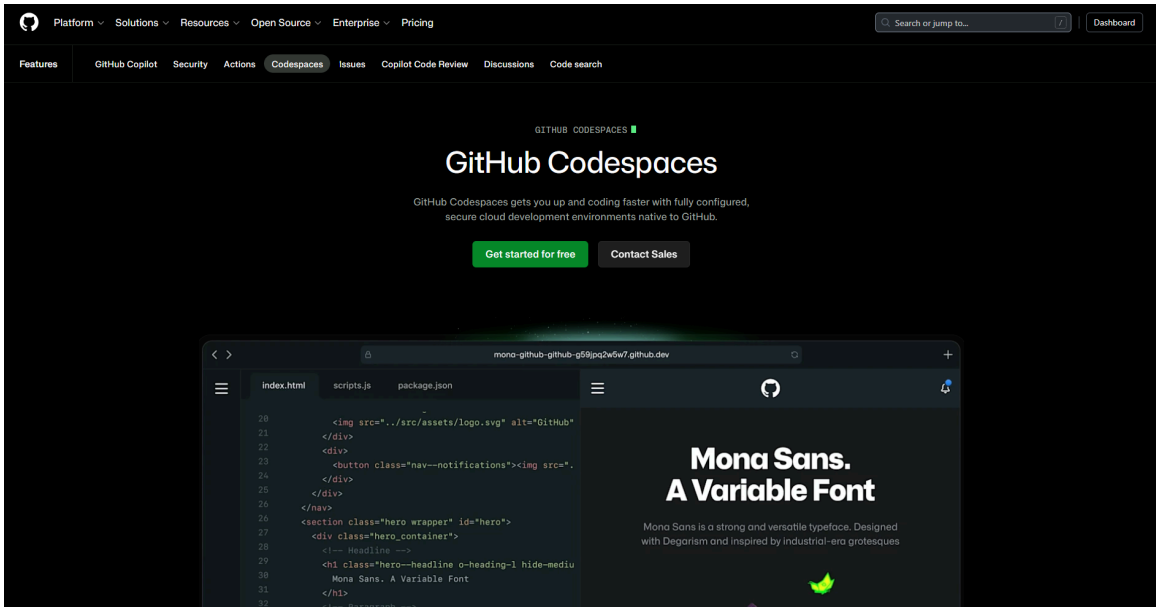
Prerequisites:

1.) Ensure that you are logged in to your GitHub account.
2.) Ensure that the deployment files are available in your GitHub repository.

Step 0.1: Open the GitHub Codespaces Page

Go to github.com/features/codespaces in your browser. This is the main landing page for GitHub Codespaces. You will see an overview of what Codespaces offers, which is basically a fully configured cloud development environment that runs right inside your browser.

Click on "Get started for free".



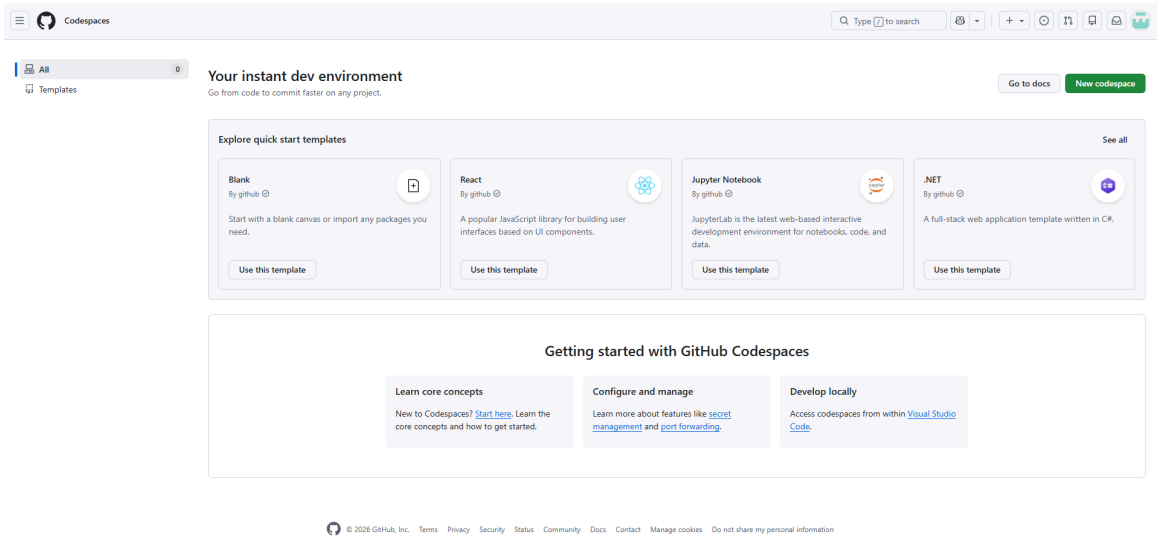
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 0.2: View Your Codespaces Dashboard

Once you are on the Codespaces dashboard, you will see a page titled "Your instant dev environment." If this is your first time, there will be no active codespaces listed.

You will also notice some quick start templates like Blank, React, Jupyter Notebook, and .NET. These are handy for starting from scratch, but since we are deploying an existing Streamlit app from a repository, we will create a new codespace from a repo instead.

Click the green "New codespace" button in the top right corner.

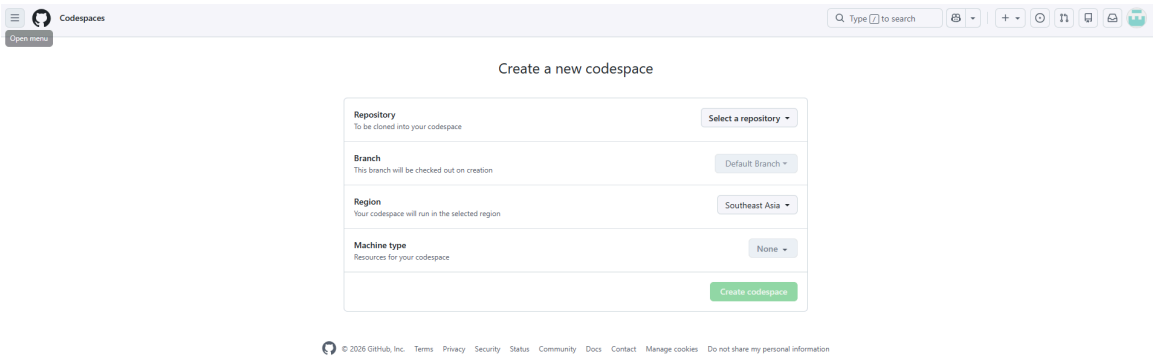


Step 0.3: Start Creating a New Codespace

You will be taken to the "Create a new codespace" page. Here you need to fill in a few details. The form has fields for Repository, Branch, Region, and Machine type.

Start by clicking the "Select a repository" dropdown and search for or select the repository that contains your Streamlit app.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 0.4: Configure and Create the Codespace

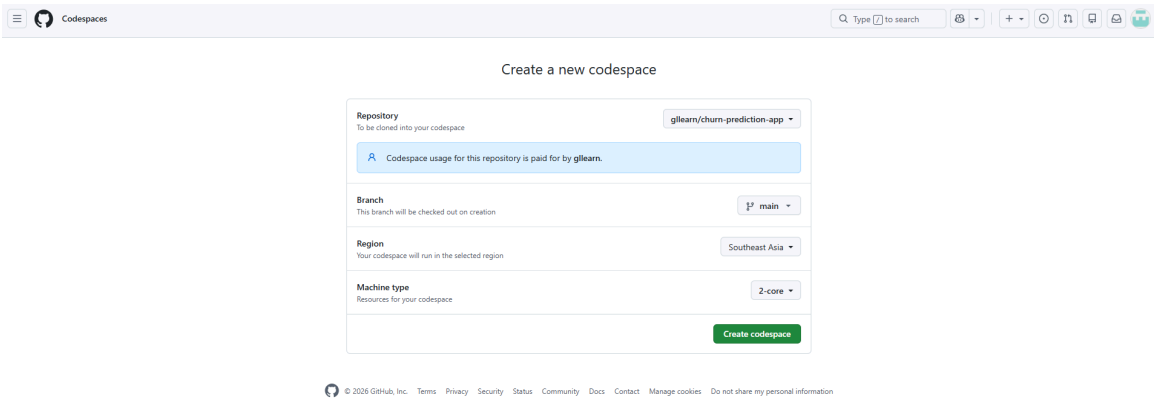
After selecting your repository (in this example, gllearn/churn-prediction-app), the form will automatically fill in some defaults. The branch will be set to "main," the region will be picked based on your location (here it shows Southeast Asia), and the machine type will default to 2-core.

You can leave the defaults as they are for most use cases. The 2-core machine is enough for running a Streamlit app. If your app is heavier, you can always bump this up later.

Note: With a 2-core machine, you will get 60 hours of usage per month, whereas with a 4-core machine, you will get 30 hours of usage per month. We highly recommend sticking with the 2-core machine, as it is more than sufficient for this case study.

Click the green "Create codespace" button to spin up your environment.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 0.5: Wait for the Codespace to Load

GitHub will now build and open your codespace. This takes a minute or two depending on the repository size. You will see VS Code loading directly in your browser.

Once it is ready, the file explorer on the left will show all the files from your repository.

In this case, you can see app.py (the main Streamlit script), churn_prediction_model_v1_0.joblib (the saved ML model), README.md, and requirements.txt.

There will be four tabs opened at the bottom: **PROBLEMS**, **OUTPUT**, **TERMINAL**, and **PORTS**. Select **TERMINAL** if it is not selected by default.

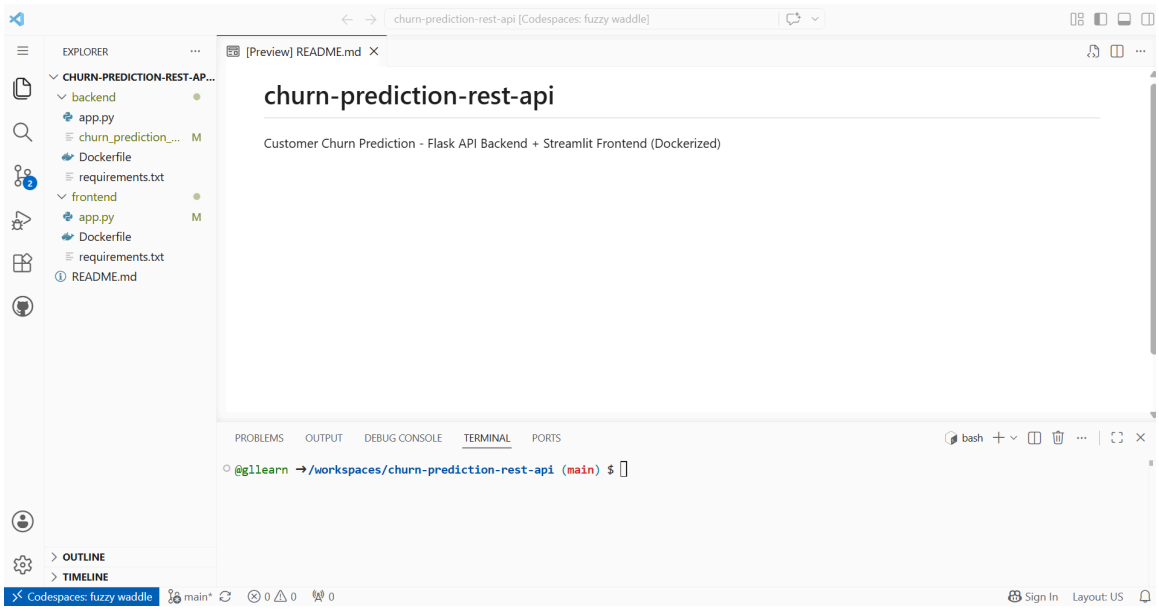
Step 1: Open the Codespace and Explore the Project Structure

Once your codespace is up and running, you will see VS Code open in your browser with the project files on the left side. The repository is called churn-prediction-rest-api and it has two main folders: backend and frontend.

The backend folder contains the Flask API (app.py, the trained model file, a Dockerfile, and requirements.txt). The frontend folder contains the Streamlit app (app.py, Dockerfile, and requirements.txt). There is also a README.md at the root.

The terminal at the bottom is where you will run all the Docker commands. You should see a fresh prompt ready to go.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

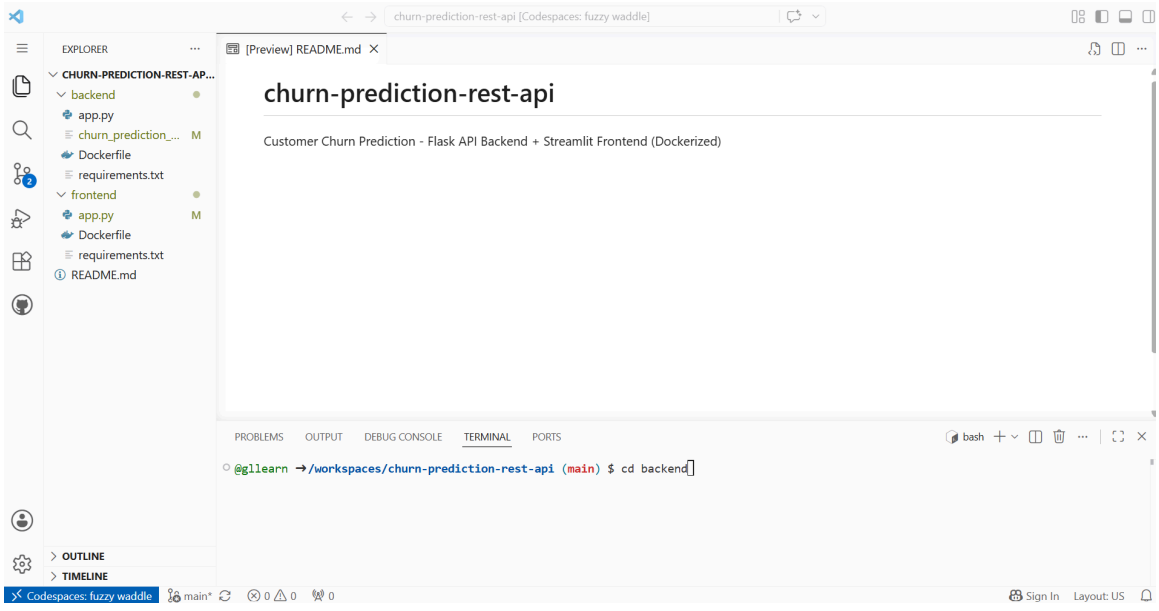


Step 2: Navigate to the Backend Directory

Before building anything, you need to move into the backend folder first. Type the following command in the terminal:

```
cd backend
```

This puts you inside the backend directory where the Dockerfile for the Flask API lives.



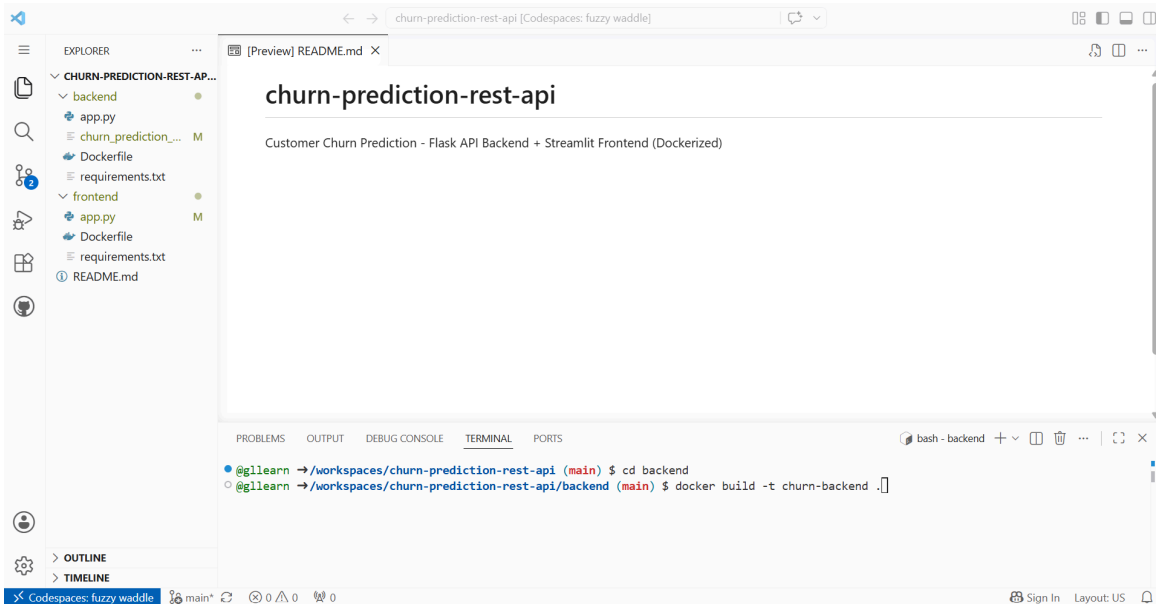
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 3: Build the Backend Docker Image

Now build the Docker image for the backend. This image will contain the Flask API and the trained churn prediction model. Run:

```
docker build -t churn-backend .
```

The -t flag tags the image with the name "churn-backend" so you can reference it later when running the container. The dot at the end tells Docker to use the Dockerfile in the current directory.

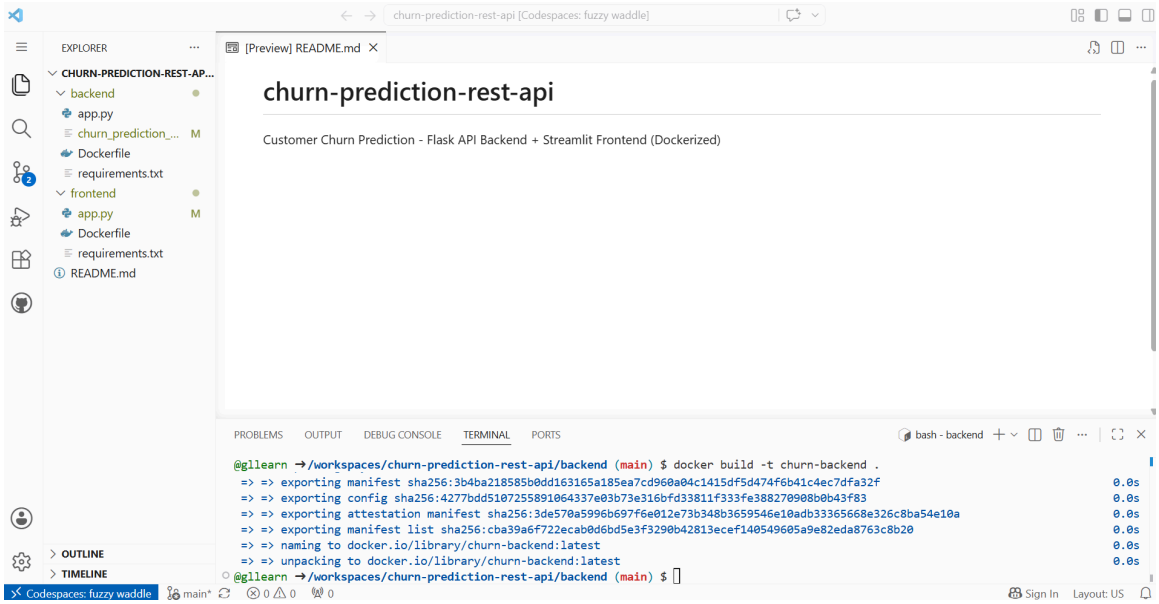


Step 4: Backend Image Build Completes

Docker will pull the base image, install all the Python dependencies from requirements.txt, and package everything up. When it is done, you will see output showing the image being named and unpacked as docker.io/library/churn-backend:latest.

This means the backend image was built successfully and is ready to be used.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



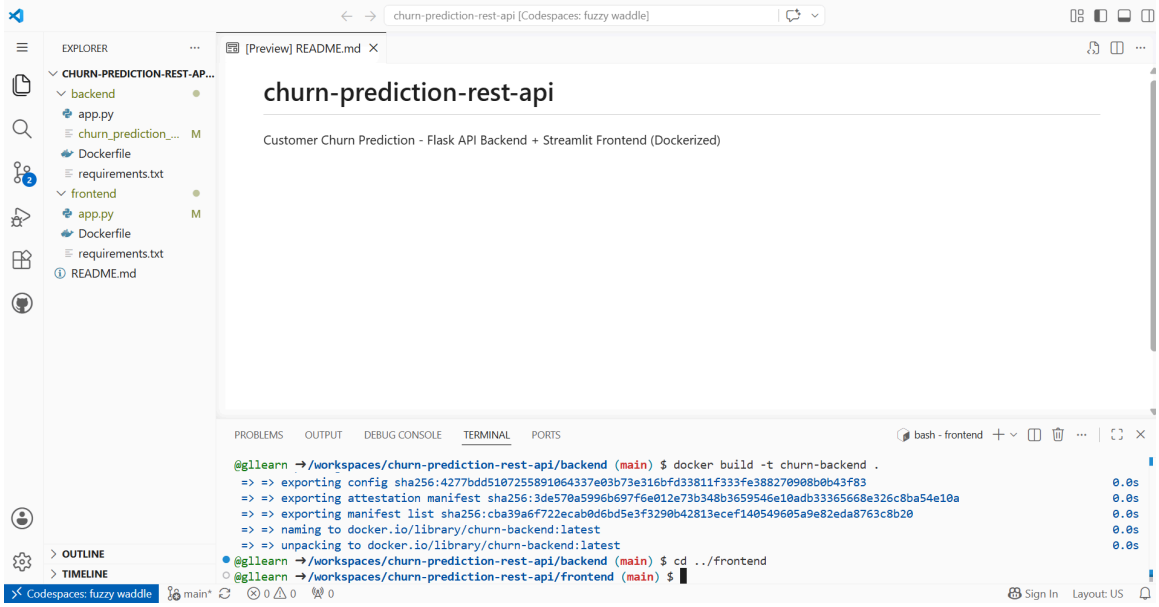
Step 5: Navigate to the Frontend and Build the Frontend Image

Next, move over to the frontend folder and build the Streamlit app image. Run the following two commands, one after the other:

```
cd ../frontend

docker build -t churn-frontend .
```

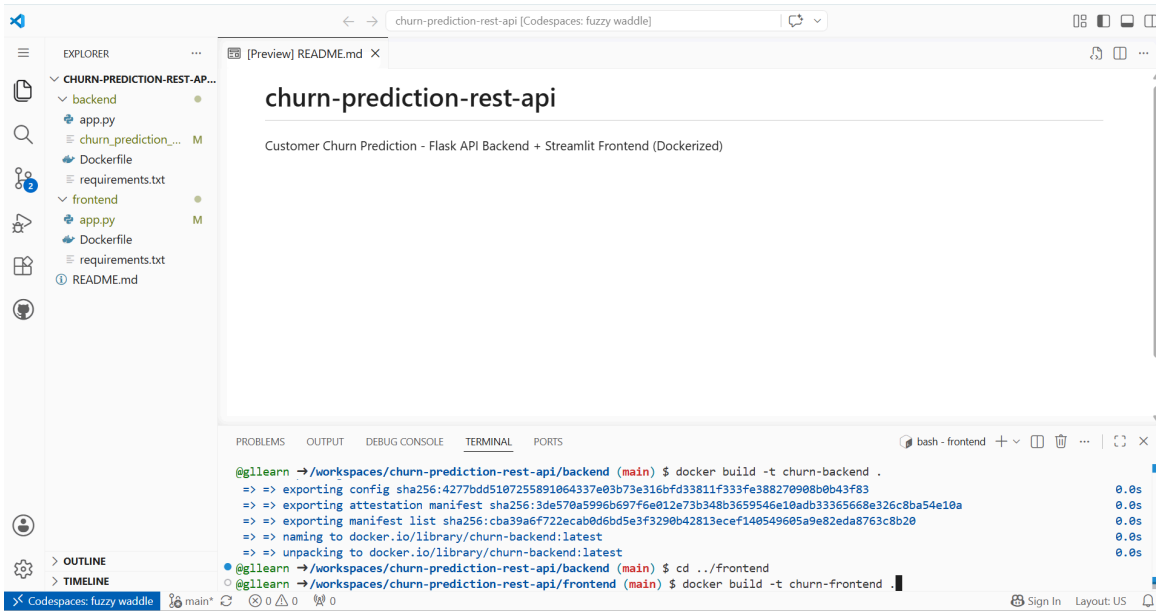
The first command takes you from the backend directory up one level and into the frontend directory. The second command builds the frontend Docker image and tags it as "churn-frontend."



This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The frontend build will go through a similar process as the backend. Docker installs Streamlit and all the other dependencies listed in the frontend requirements.txt.

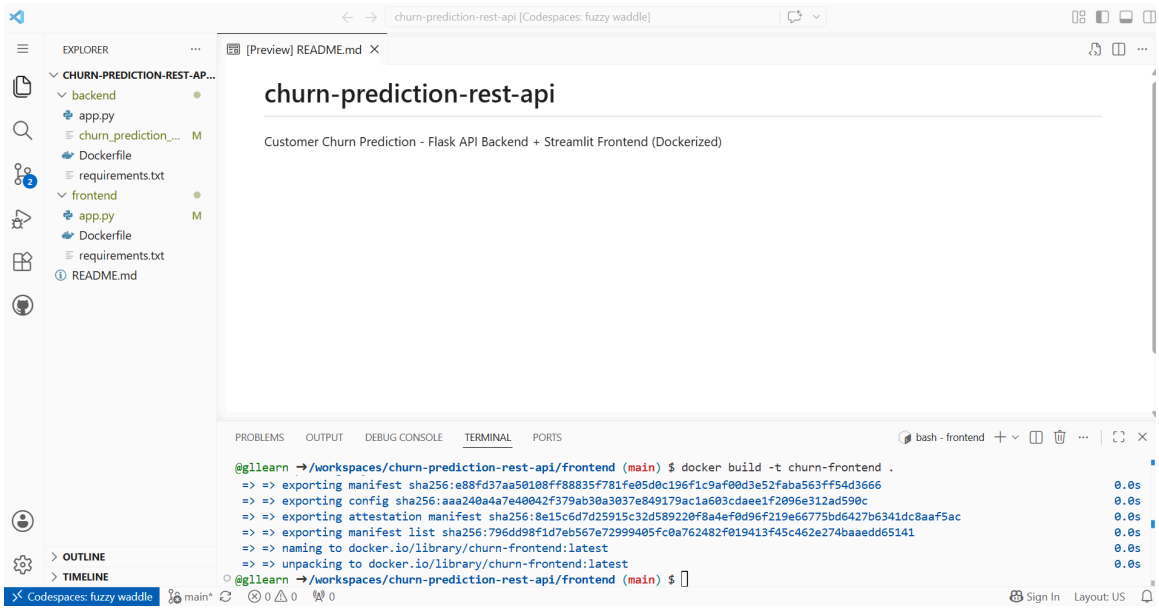
You will see the same kind of output showing layers being exported, manifests being created, and the image being named as docker.io/library/churn-frontend:latest.



Step 6: Frontend Image Build Completes

Once the frontend build finishes, you will see the full output confirming that the image was successfully created. The terminal will show the exporting steps, manifest hashes, and the final naming and unpacking lines.

At this point, both Docker images (churn-backend and churn-frontend) are built and ready to run.



This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

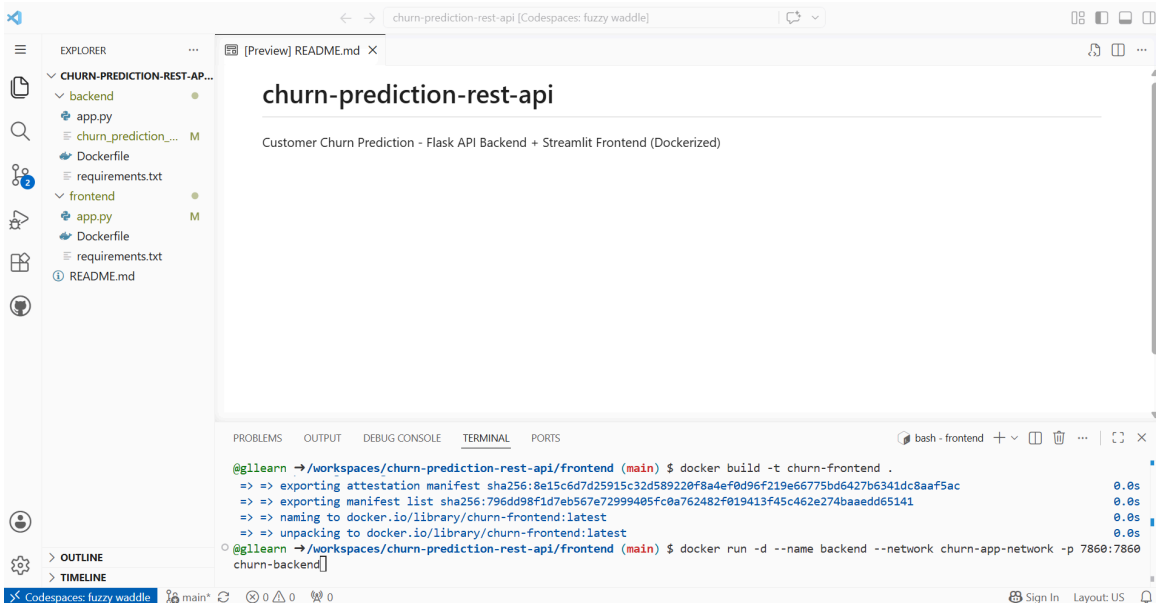
Step 7: Create a Docker Network and Run the Backend Container

Before running the containers, you need to create a Docker network so the frontend can communicate with the backend. Then start the backend container. Run the following two commands, one after the other:

```
docker network create churn-app-network

docker run -d --name backend --network churn-app-network -p 7860:7860 churn-backend
```

The first command creates a bridge network called "churn-app-network." The second command runs the backend container in detached mode (-d), names it "backend," connects it to the network, and maps port 7860 from the container to port 7860 on the host.

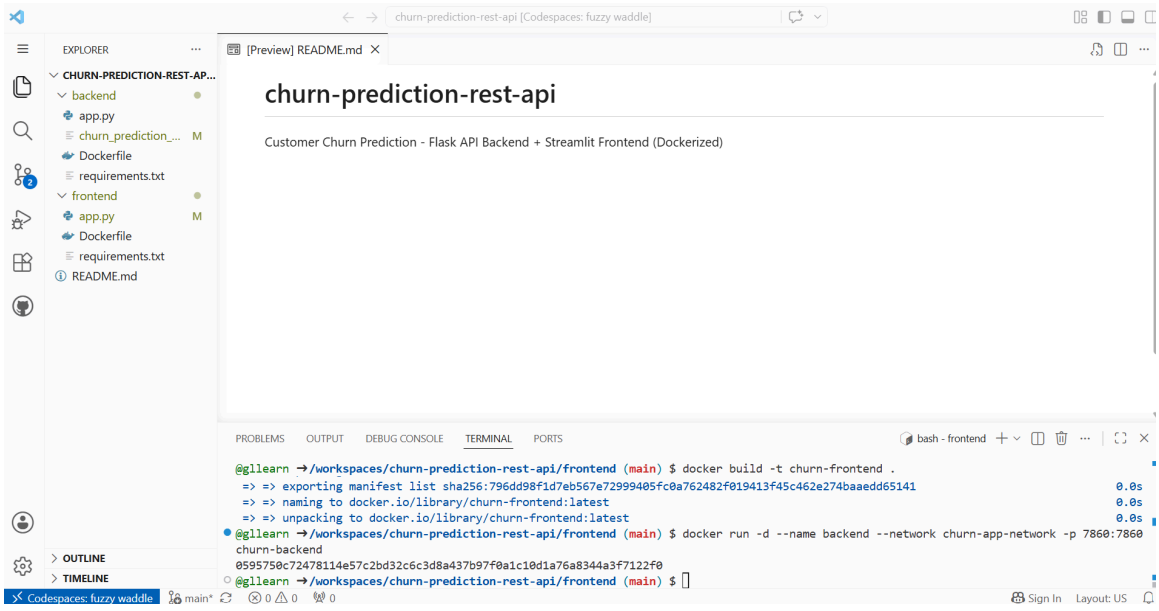


Step 8: Backend Container Starts Successfully

After running the docker run command, you will see a long container ID hash printed in the terminal. This confirms that the backend container has started and is running in the background.

The Flask API is now listening on port 7860 inside the container, and that port is mapped to your codespace.

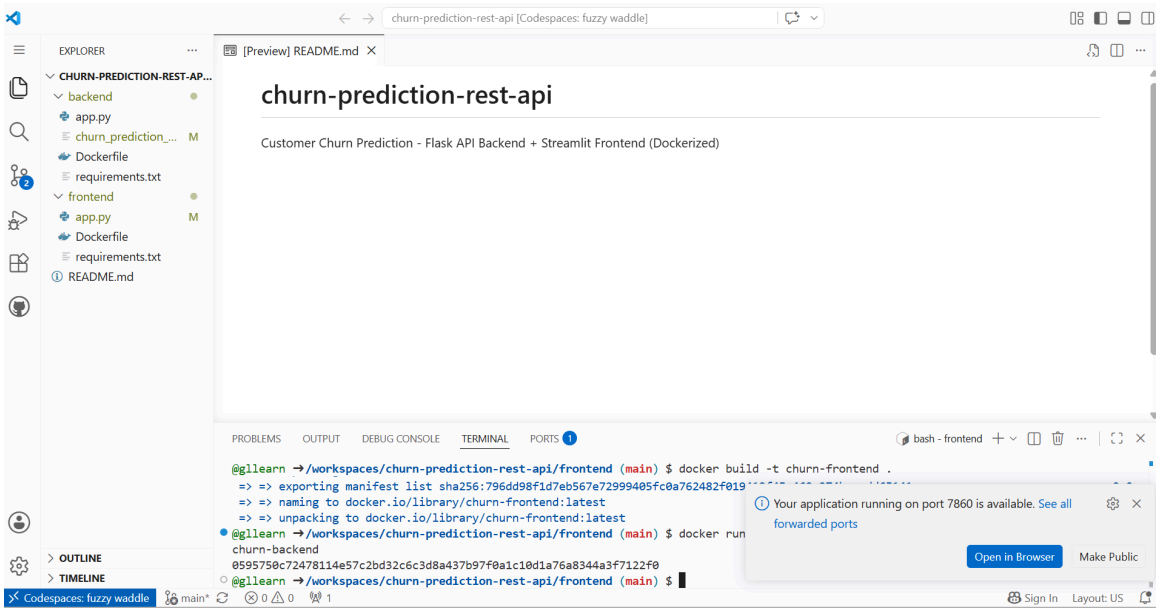
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 9: Port 7860 Notification Appears

Shortly after the backend container starts, a notification will pop up in the bottom right corner of VS Code saying "Your application running on port 7860 is available." You will see "Open in Browser" and "Make Public" buttons.

Click "Make Public" so the backend API is accessible. You do not necessarily need to open it in the browser since the frontend will call it internally through the Docker network, but making it public ensures port forwarding works correctly.



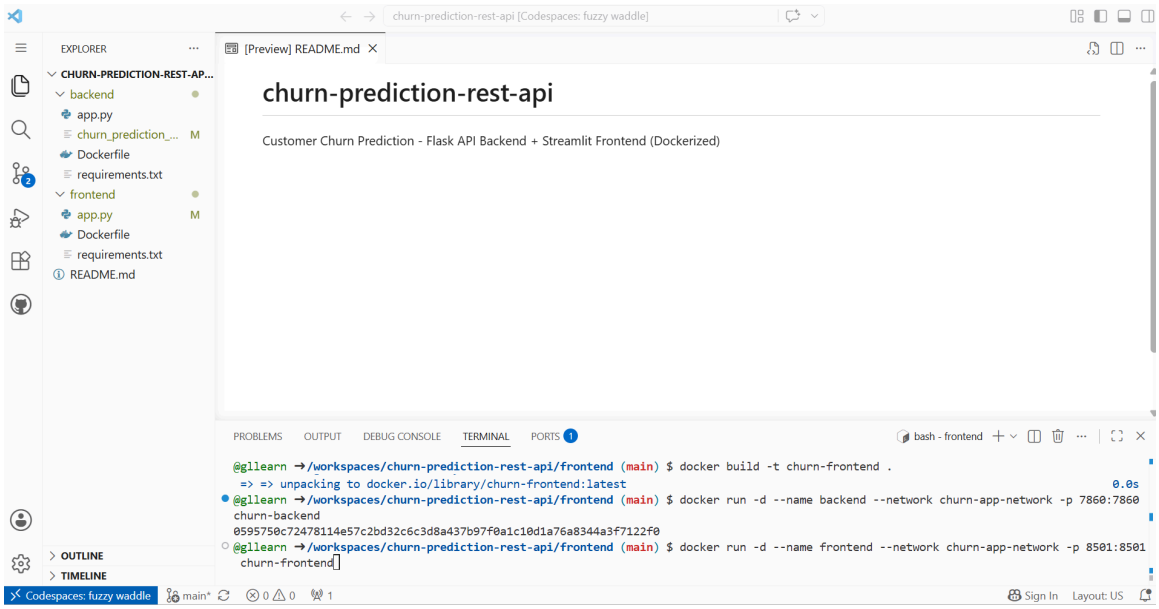
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 10: Run the Frontend Container

Now run the frontend container on the same network. Type:

```
docker run -d --name frontend --network churn-app-network -p 8501:8501 churn-frontend
```

This starts the Streamlit frontend in detached mode, names it "frontend," connects it to the same churn-app-network, and maps port 8501. Because both containers share the same Docker network, the frontend can reach the backend using the container name "backend" as the hostname.

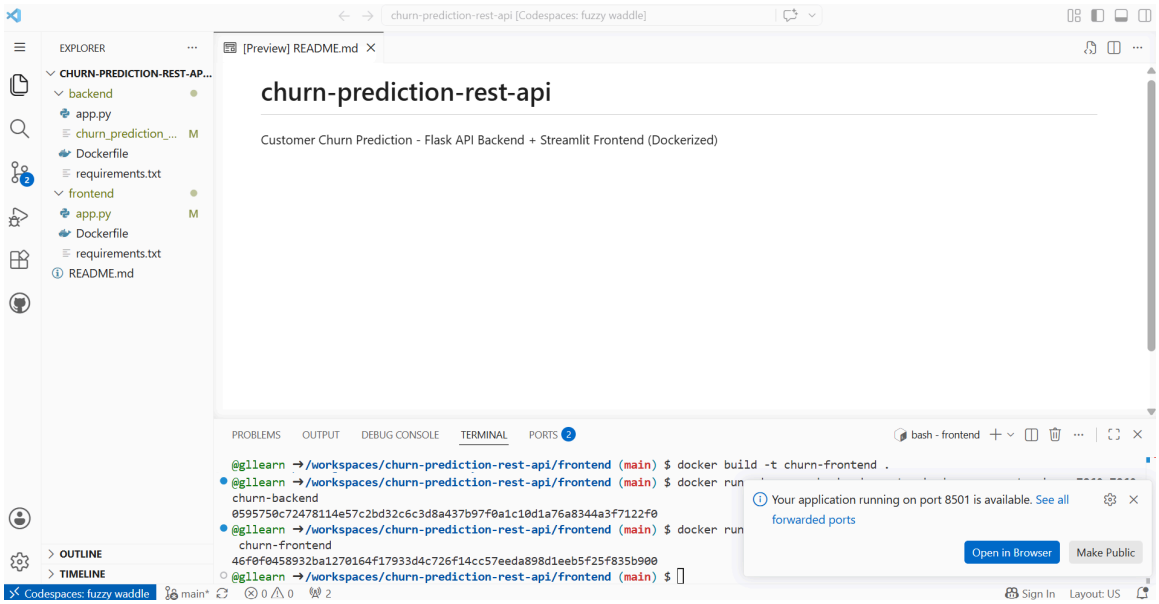


Step 11: Port 8501 Notification Appears

Once the frontend container starts, another notification will pop up saying "Your application running on port 8501 is available." The PORTS tab now shows 2, meaning both ports are being forwarded.

Click "Make Public" and then "Open in Browser" to access the Streamlit frontend.

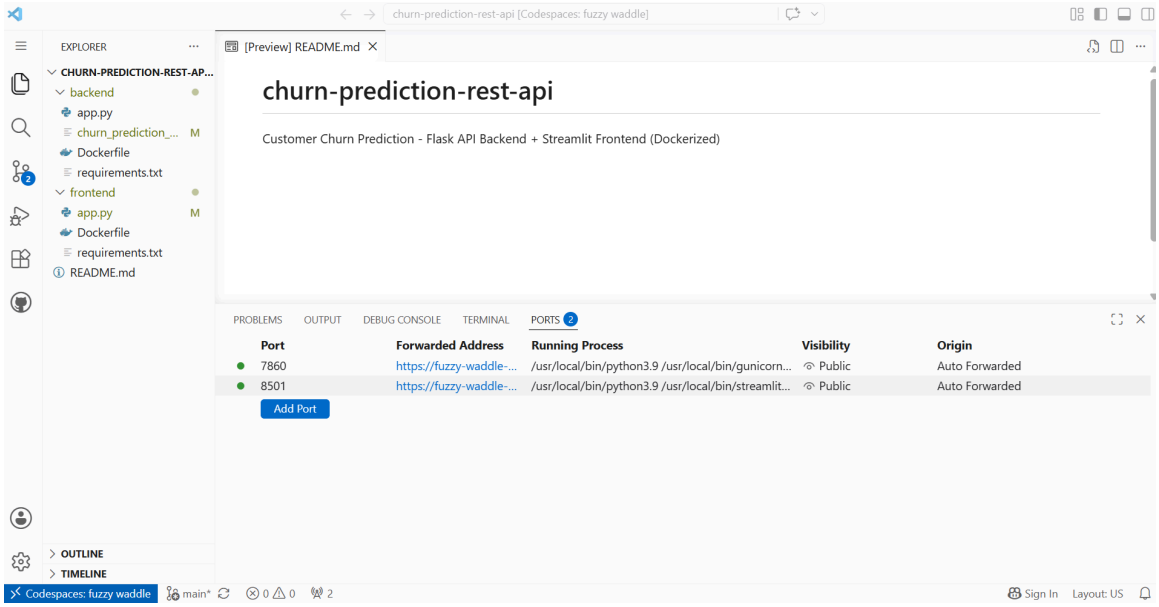
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 12: Verify Both Ports Are Forwarded and Public

Click on the PORTS tab at the bottom of VS Code to see all forwarded ports. You should see two entries: port 7860 running the gunicorn/Flask backend, and port 8501 running the Streamlit frontend.

Make sure both ports have their Visibility set to "Public."Both services need to be publicly accessible for the app to work properly from your browser.

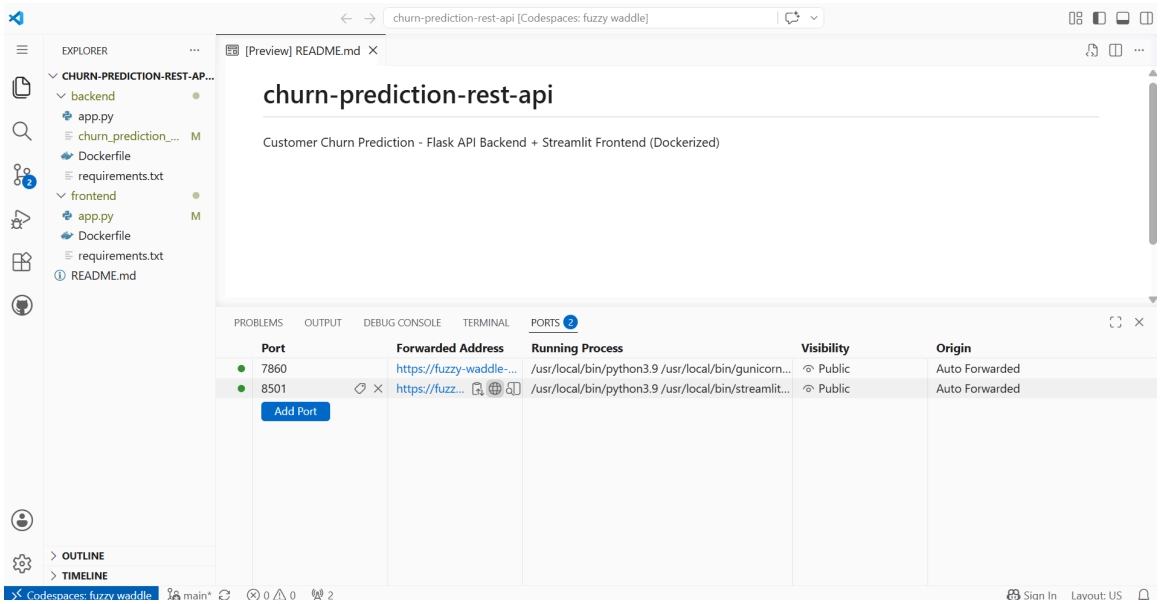


Step 13: Confirm Both Services Are Running

You can also see the forwarded addresses for each port in the Ports panel. The backend API (port 7860) and the Streamlit frontend (port 8501) each get their own unique URL from GitHub Codespaces.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Copy the forwarded address for **port 7860** and save it somewhere for future reference.

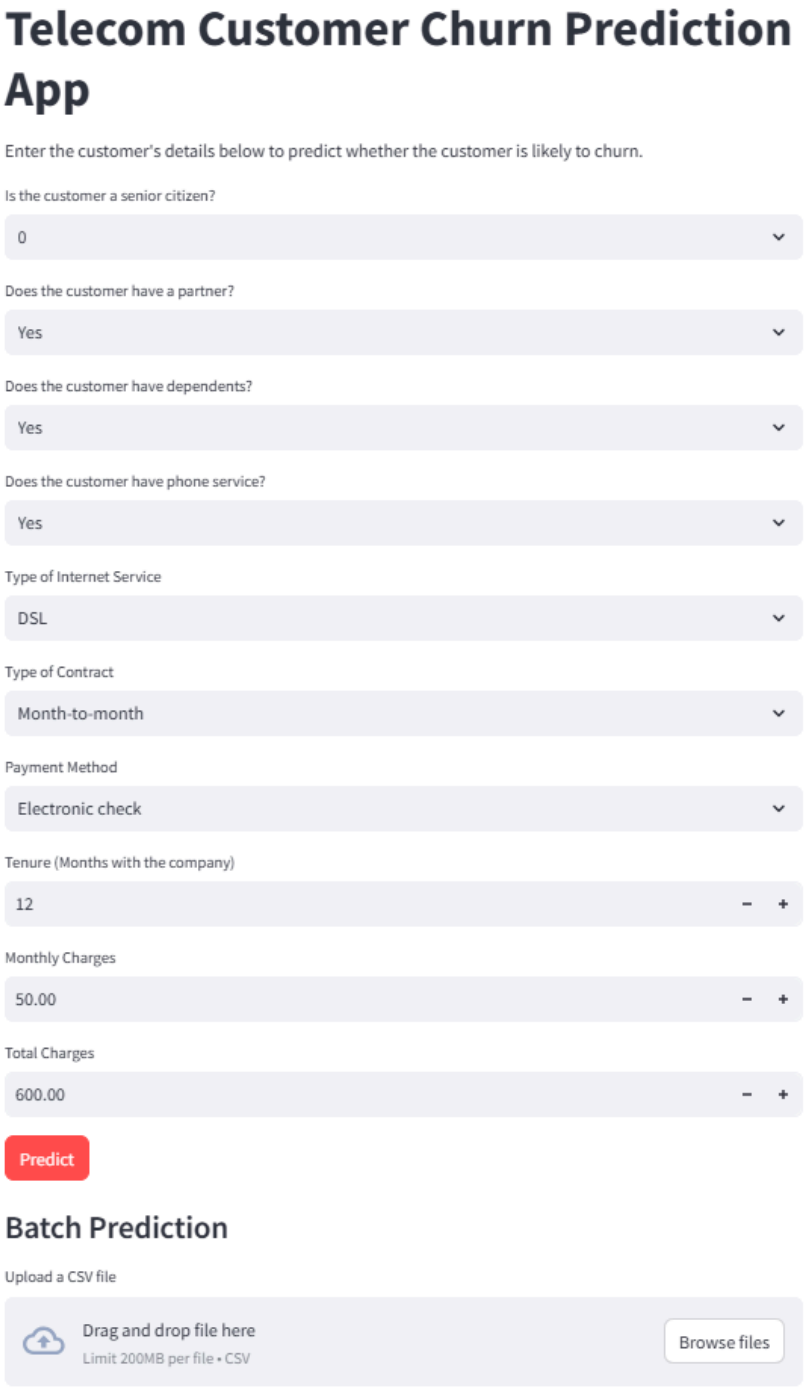


Step 14: The Telecom Churn Prediction App Is Live

When you open the port 8501 URL in your browser, you will see the Telecom Customer Churn Prediction App. The page has a form at the top for single customer predictions with fields like senior citizen status, partner, dependents, phone service, internet service type, contract type, payment method, tenure, monthly charges, and total charges.

Below the form there is a "Predict" button and further down you will find a "Batch Prediction" section where you can upload a CSV file for bulk predictions.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

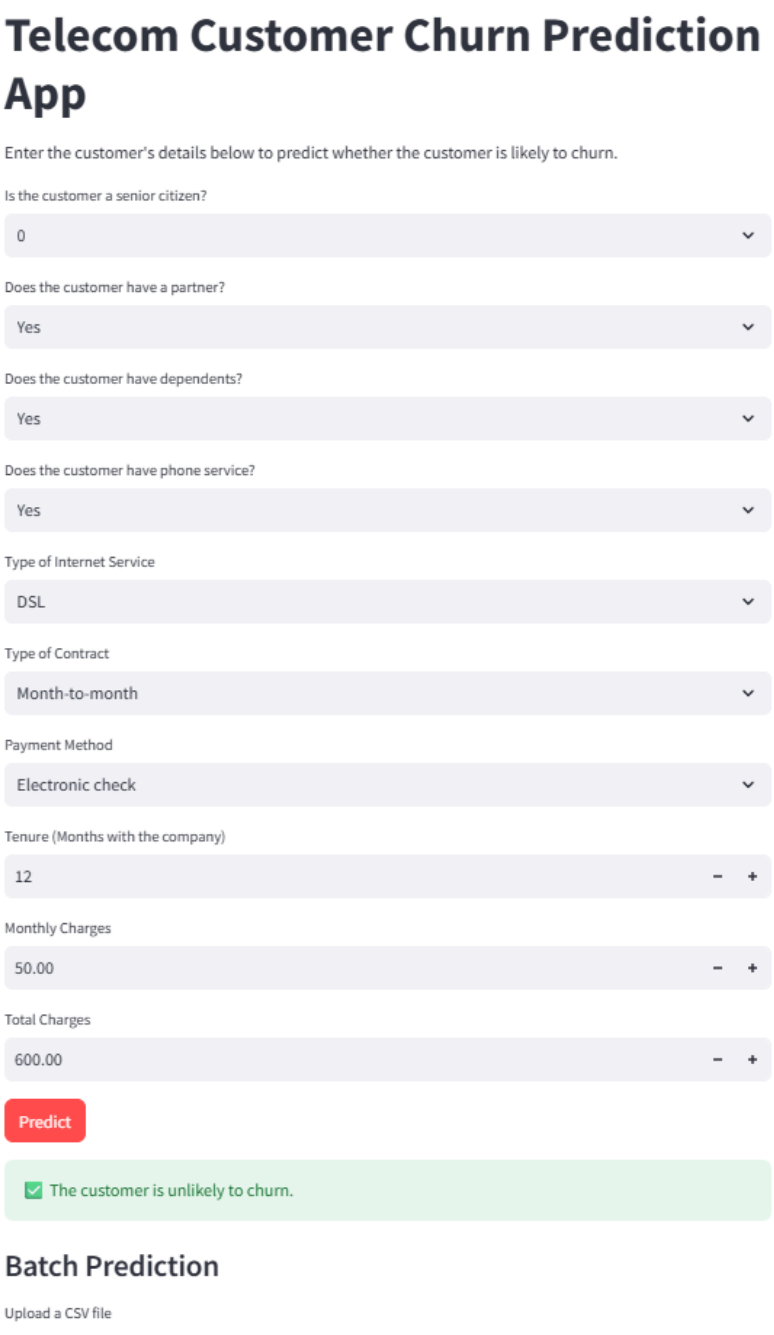


Step 15: Test a Single Customer Prediction

Fill in the form fields with some sample values and click the red "Predict" button. The app will send the data to the Flask backend API, which runs it through the trained model and returns the prediction.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

In this example, with the values shown (not a senior citizen, has a partner and dependents, has phone service, DSL internet, month-to-month contract, electronic check payment, 12 months tenure, 50.00 monthly charges, 600.00 total charges), the result comes back as "The customer is unlikely to churn." The green check mark confirms a successful prediction.



This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 16: Test Batch Prediction with a CSV File

Scroll down to the Batch Prediction section and click "Browse files" or drag and drop a CSV file containing multiple customer records. In this example, a file called Batch-data-telecom.csv (0.9KB) has been uploaded.

Click the red "Predict for Batch" button. The app sends all the records to the backend API at once, and the results come back as a table showing the prediction for each customer (Churn or Not Churn).

A green banner confirms "Predictions completed successfully!" and you can scroll through the results table to see the outcome for each customer ID.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Yes

Does the customer have phone service?

Yes

Type of Internet Service

DSL

Type of Contract

Month-to-month

Payment Method

Electronic check

Tenure (Months with the company)

12

Monthly Charges

50.00

Total Charges

600.00

Predict

Batch Prediction

Upload a CSV file

Drag and drop file here

Limit 200MB per file • CSV

Browse files

Batch-data-telecom.csv 0.9KB

Predict for Batch

Predictions completed successfully!

	CUST0048	CUST0049	CUST0050	CUST0051	CUST0052	CUST0053	CUST0054	CUST0055	CUST
0	Not Churn	Churn	Not Churn	Churn	Churn	Not Churn	Churn	Not Churn	Churn

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 17: View Your Active Codespace in the Dashboard

If you go back to github.com/features/codespaces, you will see your newly created codespace listed under **"Your codespaces."** It shows the repository name, the codespace name (in this case "studious spoon"), the branch (main), machine specs (2-core, 8GB RAM, 32GB), disk usage, and the status (Active).

From here you can click on the codespace name to reopen it in the browser at any time.

Step 18: Manage Your Codespace

Click the three dots on the right side of your codespace entry to see management options. You can rename it, export changes to a branch, change the machine type, or stop the codespace when you are done working.

Please make sure to stop the codespace when you are not using it. You can also enable auto-delete so it gets cleaned up automatically after a period of inactivity.

Note: As of July 14, 2026, GitHub Codespaces provides 60 hours of usage per month for a 2-core CPU configuration. Hence, it is highly advisable to stop the Codespace whenever it is not in use to maximize the available usage hours and get the maximum benefit from the free-tier allowance.

Wrapping Up

That covers the full deployment process. To recap: you build separate Docker images for the backend and frontend, create a shared Docker network so the two containers can talk to each other, run both containers with the right port mappings, make the ports public in Codespaces, and you are good to go.

The backend API handles the ML predictions and the frontend gives users a clean interface to interact with it. If you need to make changes, just stop the containers with "docker stop backend frontend," rebuild the images, and run them again.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.